Programmation Orientée Objet (POO) à destination de la robotique

Travaux Pratiques : Navigation Autonome (A* et DWA)

Enseignant : Adam GOUGUET
Mail : adam.gouguet@imt-nord-europe.fr
Année : 2025–2026

Table des matières

1 Objectifs du TP **1**

2 Planification Globale avec A* **2**
2.1 Structure de la classe 2

3 Contrôle Local avec PID **3**
3.1 Principe du suivi de chemin 3
3.2 Commande linéaire 3
3.3 Commande angulaire 3
3.4 Pseudo-code du contrôleur PID 4

4 Intégration (La Stack de Navigation) **5**

5 Pour aller plus loin (Bonus) **5**

1 Objectifs du TP

En robotique mobile, la navigation autonome est généralement scindée en deux composants ayant des **responsabilités distinctes** :

— **Le Planificateur Global (A*)** : Il calcule un chemin approximatif entre le robot et sa destination finale en utilisant la carte complète. Il s'exécute à basse fréquence (ex : 1 Hz ou uniquement quand on donne un nouvel objectif).

— **Le Contrôleur Local (PID - Proportionnel, Intégral, Dérivé)** : Il gère la cinématique du robot (vitesse linéaire et angulaire) et l'évitement réactif des obstacles imprévus. Il suit le chemin global tout en s'exécutant à haute fréquence (ex : 20 Hz).

Nous allons implémenter ces deux comportements dans des classes séparées pour respecter les principes *SOLID* (Responsabilité Unique).

2 Planification Globale avec A*

Créez un fichier `planificateur_global.py`. La classe `PlanificateurAStar` va prendre une `GrilleOccupation` par **Injection de Dépendance** (composition).

> Si vous n'avez pas fait le TP sur la cartographie, aller voir comment implémenter la `GrilleOccupation`.

2.1 Structure de la classe

L'algorithme A* (A-étoile) fonctionne sur la grille discrète. Il va évaluer les cellules voisines en utilisant un coût réel (g) et une heuristique (h).

```
import heapq

class PlanificateurAStar:
    def __init__(self, grille):
        self.grille = grille

    def heuristique(self, idx_a, idx_b):
        """ Calcule la distance de Manhattan ou Euclidienne entre deux indices """
        # TODO : Implmenter le calcul de distance
        pass

    def trouver_chemin(self, depart_m, arrivee_m):
        """
        Calcule le chemin de 'depart_m' (x,y) vers 'arrivee_m' (x,y) en metres.
        Retourne une liste de points (x, y) en metres.
        """
        # 1. Convertir les coordonnees metriques en indices de grille
        start_idx = self.grille.coord2index(*depart_m)
        goal_idx = self.grille.coord2index(*arrivee_m)

        # 2. Verifier que l'arrivee n'est pas dans un obstacle (etat == 1)
        if self.grille.get_cellule(*goal_idx) == 1:
            print("Erreur : L'objectif est dans un obstacle !")
            return []

        # TODO : Initialiser la file de priorite (open_set) et les dictionnaires
            g_score et came_from

        # TODO : Boucle principale A*
        # Indice : Pour recuperer les voisins valides, verifiez qu'ils ne sortent pas
        # de la grille et que grille.get_cellule(nx, ny) != 1.

        # 3. Reconstruire le chemin (liste d'indices)

        # 4. Convertir le chemin d'indices (ix, iy) en coordonnees monde (x, y)
        chemin_metres =[]
        # TODO : Remplir chemin_metres
```

```
        return chemin_metres
```

Travail à faire : Complétez les méthodes de la classe. N'oubliez pas que votre algorithme A* travaille avec les indices de la grille, mais **doit retourner des coordonnées en mètres** pour que le contrôleur local (qui travaille dans le monde physique) puisse les utiliser !

3 Contrôle Local avec PID

Nous allons ici implémenter un **contrôleur proportionnel (PID simplifié)** pour suivre le chemin généré par A*.

Le contrôleur PID calcule directement une vitesse linéaire v et angulaire ω à partir de l'erreur entre la position actuelle du robot et le point cible.

Nous utiliserons uniquement la composante **proportionnelle (P)** pour simplifier l'implémentation.

3.1 Principe du suivi de chemin

Le chemin généré par A* est une liste de points :

$$P = \{(x_1, y_1), (x_2, y_2), ..., (x_n, y_n)\}$$

Le contrôleur vise un point à la fois.
À chaque itération :

1. On sélectionne le point courant (x_c, y_c).
2. On calcule l'erreur de position :
$$dx = x_c - x$$
$$dy = y_c - y$$
3. On calcule la distance euclidienne :
$$d = \sqrt{dx^2 + dy^2}$$

3.2 Commande linéaire

La vitesse linéaire est proportionnelle à la distance au point cible :

$$v = K_p^{lin} \cdot d$$

Plus le robot est loin du point, plus il avance vite.

Lorsque d devient inférieur à une tolérance ϵ, on considère le point atteint et on passe au suivant.

3.3 Commande angulaire

Le robot doit également s'orienter vers la cible.

On calcule l'angle désiré :

$$\theta_{des} = \arctan 2(dy, dx)$$

L'erreur angulaire est :

$$e_\theta = \theta_{des} - \theta$$

Cette erreur doit être normalisée dans l'intervalle $[-\pi, \pi]$ pour éviter les discontinuités. La vitesse angulaire est alors :

$$\omega = K_p^{ang} \cdot e_\theta$$

3.4 Pseudo-code du contrôleur PID

```
Entres :
   - obs = {pose (x,y), orientation theta}
   - path = liste de points [(x1,y1), ..., (xn,yn)]
   - index = indice du waypoint courant
   - dt = pas de temps

Sortie :
   - Commande (v, omega)

Algorithme :

1. Si index >= taille(path)
      retourner (0, 0)

2. Extraire le waypoint courant (x_c, y_c)

3. Calculer l erreur de position :
      dx = x_c - x
      dy = y_c - y
      distance = sqrt(dx^2 + dy^2)

4. Si distance < tolerance
      index = index + 1
      retourner (0, 0)

5. Calculer l angle desire :
      theta_des = atan2(dy, dx)

6. Calculer l erreur angulaire :
      e_theta = theta_des - theta
      normaliser e_theta dans [-pi, pi]

7. Calculer les commandes proportionnelles :
      v = Kp_lin * distance
      omega = Kp_ang * e_theta
```

```
8. Retourner (v, omega)
```

Le contrôleur PID est simple à implémenter et très rapide à exécuter. Cependant, il ne prend pas en compte explicitement les obstacles dynamiques. C'est pourquoi les architectures modernes utilisent souvent un contrôleur prédictif (DWA, MPC, NMPC) au lieu d'un simple PID.

4 Intégration (La Stack de Navigation)

Dans votre fichier `main.py`, vous allez maintenant lier les deux algorithmes. Le `PlanificateurAStar` va générer une liste de points (ex : 100 points). Le `PID` ne doit pas viser le point d'arrivée final, mais un **point cible local** (ex : le 10ème point de la liste A* devant le robot, que l'on appelle le point *Look-ahead*).

Travail à faire (Pseudo-code du main.py) :

1. Récupérer la carte via la `GrilleOccupation`.
2. Calculer le chemin A* une seule fois (avant la boucle while).
3. Dans la boucle du jeu/simulation :
   — Extraire la "cible locale" (le point du chemin A* situé à 1 mètre du robot).
   — Lire les points du Lidar.
   — Appeler `pid.calculer_commande()`.
   — Appliquer (v, ω) au robot virtuel.

5 Pour aller plus loin (Bonus)

1. **Lissage de trajectoire :** Le chemin généré par A* sur une grille est très anguleux (en "escaliers"). Implémentez un algorithme de lissage (ex : descente de gradient, ou B-spline) dans la classe globale.
2. **Récupération (Recovery Behavior) :** Que se passe-t-il si le DWA ne trouve aucune trajectoire valide (toutes mènent à une collision) ? Implémentez une commande de repli (ex : rotation sur place $\omega > 0, v = 0$ pour chercher une issue, ou recalcul total du A*).
3. **Implémenter d'autres Planificateur/Contrôleur :**
   — Planificateur : baser sur un graphe (D*/theta*), planificateurs probabilistes (Probabilistic Roadmap, Rapidly-exploring Random Tree, ...), planificateurs d'optimisation (Optimisation quadratique, Méthodes de gradient, ...).
   — Controleur : Dynamic Window Approach, Model Predictive Control, Timed-based Elastic Band, ...